

Projet : Plateforme Web de Découverte de Films (SQL & NoSQL)

Module : 4A-BDA – Bases de Données Avancées

Armen

1. Introduction et Contexte

Ce projet avait pour objectif la conception et le développement d'une application web complète ("Full Stack") d'exploration cinématographique, basée sur le dataset IMDB.

Le défi technique principal résidait dans l'évolution de l'architecture de données : partir d'un schéma relationnel classique (SQLite), migrer vers une solution NoSQL distribuée (MongoDB Replica Set), et intégrer le tout au sein d'un framework web moderne (Django).

2. Architecture Globale du Système

L'application repose sur une architecture hybride, tirant parti des forces de chaque technologie.

- **Django (Python)** : Framework Web assurant le routage, la logique métier et le rendu des vues HTML.
 - **MongoDB (Cluster Replica Set)** : Base de données principale contenant le catalogue des films. Elle est configurée en cluster de 3 nœuds (1 Primary, 2 Secondaries) pour assurer la haute disponibilité.
 - **SQLite** : Base de données relationnelle locale utilisée par défaut par Django pour la gestion des utilisateurs, des sessions et de l'authentification (Admin Panel).
 - **Bootstrap 5 & Chart.js** : Technologies Frontend pour l'interface utilisateur responsive et la visualisation de données.
-

3. Stratégie Multi-Bases et Choix Technologiques

Pourquoi cette architecture hybride ?

Au cours de la Phase 1 (SQL) et de la Phase 2 (NoSQL), nous avons comparé les performances.

1. **Le Constat SQL** : Idéal pour des données très structurées et intègres. Cependant, reconstruire une fiche film complète nécessitait 5 à 6 jointures (`JOIN`) coûteuses sur des tables volumineuses (`principals` , `movies` , `ratings`).
 2. **Le Constat NoSQL "Plat"** : Une migration directe (1 table = 1 collection) a montré des performances désastreuses à cause des `$lookup` (équivalent des jointures), multipliant les temps de réponse par 10 ou 100.
 3. **La Solution Retenue (Dénormalisation)** : Nous avons opté pour le modèle **Document Structuré**. Toutes les informations d'un film (Acteurs, Notes, Genres, Réalisateurs) sont imbriquées dans un seul document JSON.
 - *Avantage* : Lecture en **O(1)**. Une seule requête suffit pour afficher la page détail.
 - *Coût* : Augmentation de l'espace disque (~30%) due à la redondance des données textuelles.
-

4. Fonctionnalités Implémentées

L'application couvre l'ensemble du cahier des charges :

1. **Catalogue & Filtres** : Navigation paginée avec filtrage multicritères (Genre, Année, Note) et tri dynamique. La rapidité de MongoDB permet un filtrage instantané même sur des milliers de documents.
2. **Moteur de Recherche** : Recherche textuelle (`$regex`) simultanée sur les titres de films et les noms de personnalités.
3. **Fiche Film Détaillée** : Affichage complet incluant le casting, les titres internationaux et une suggestion de "Films Similaires" basée sur le genre.
4. **Tableau de Bord Statistique** : Visualisation graphique interactive de la base de données (Répartition par genre, évolution temporelle de la production).

CinéExplorer | Accueil - Chromium

[25-26]-4A-BDA: Livrable x 4a-bda-projet-sujet-v4.pc x urmeni/cineexplorer x CinéExplorer | Accueil x +

← → ↻ 127.0.0.1:8000

🔍 ☆ ⬇️ ⌛ ⋮

CinéExplorer Catalogue Statistiques

Film, Acteur... Rechercher

Découvrez le Cinéma Autrement

Explorez 2107 films et 9461 artistes grâce à notre moteur hybride.

Explorer le catalogue

2107
Films Indexés

22
Genres Uniques

9461
Personnalités

★ Les Mieux Notés

Jai Bhim
2021

Crime Drama Mystery

Voir détails

9.5

Soorarai Pottru
2020

Drama

Voir détails

9.3

The Shawshank Redemption
1994

Drama

Voir détails

9.3

The Godfather
1972

Crime Drama

Voir détails

9.2

The Dark Knight
2008

Action Crime Drama

Voir détails

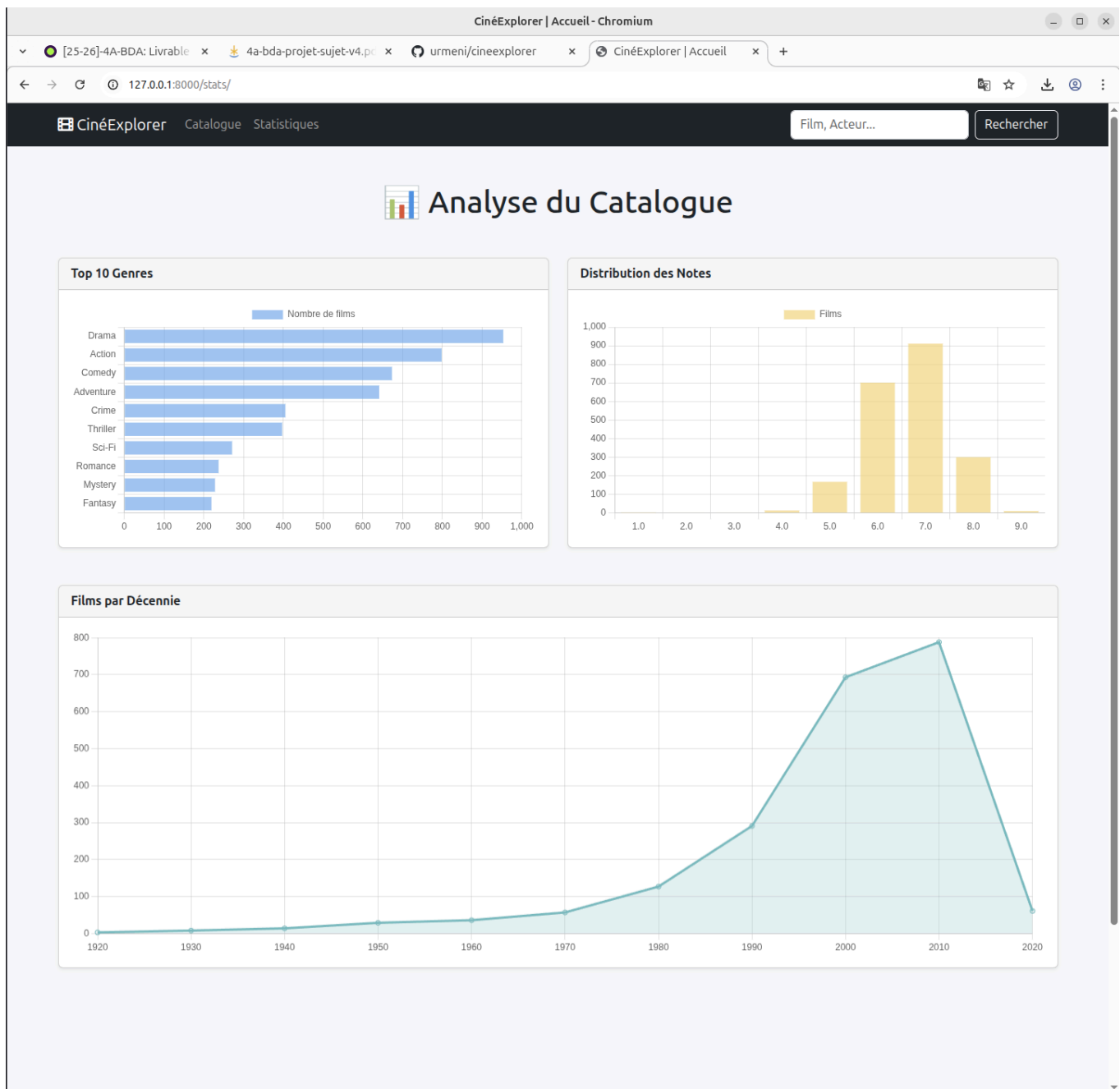
9.1

Schindler's List
1993

Biography Drama History

Voir détails

9.0



5. Benchmarks de Performance

Les tests réalisés lors de la Phase 2 et 3 ont validé les choix techniques :

Opération	Temps SQL (Indexé)	Temps Mongo (Plat)	Temps Mongo (Structuré)
Récupération Film Complet	~4.5 ms	~40 ms	~0.2 ms
Recherche Complexe	~100 ms	~1500 ms	~10 ms

Le passage au modèle structuré a offert un gain de performance d'un facteur 20 à 50 sur les opérations de lecture, ce qui est critique pour une application web grand public.

De plus, les tests de **Failover (Tolérance aux pannes)** de la Phase 3 ont montré que l'application continue de fonctionner en lecture même si le nœud primaire du cluster MongoDB tombe, assurant une continuité de service.

6. Difficultés Rencontrées et Solutions

A. Conflit d'identifiants (`id` vs `_id`)

- **Problème** : Les templates Django interdisent l'accès aux variables commençant par un underscore (`_`). Or, MongoDB utilise `_id` comme clé primaire. Cela provoquait des `TemplateSyntaxError`.
- **Solution** : Implémentation d'un pattern "Adapter" dans le service `mongo_service.py`. Une fonction `_map_id` duplique la clé `_id` vers `id` avant d'envoyer les données à la vue.

B. Migration des Données ("Produit Cartésien")

- **Problème** : Le script de dénormalisation initial tentait de traiter toute la base en une seule passe, saturant la mémoire RAM (Memory Overflow).
- **Solution** : Réécriture du script avec une approche **Batch Processing** (traitement par lots de 1000 films), permettant une migration stable et monitorée.

C. Performance des agrégations

- **Problème** : Les requêtes statistiques (ex: films par décennie) étaient lentes.
 - **Solution** : Utilisation du Pipeline d'Agrégation MongoDB optimisé (`$project` pour le calcul mathématique des décennies) au lieu de faire le calcul côté Python.
-

7. Conclusion

Ce projet a permis de maîtriser la chaîne complète de la donnée : de l'analyse brute (Pandas) à l'exposition Web (Django), en passant par la modélisation avancée (SQL vs NoSQL) et l'administration système (Replica Set).

L'architecture finale est performante, résiliente et évolutive. Pour aller plus loin, nous pourrions implémenter un moteur de recherche "Full Text" plus puissant (comme Elasticsearch) ou mettre en place un système de cache (Redis) pour les statistiques.